PGARA

Profile-Guided Adaptive Register Allocation

The complete project guide: the idea, the research behind it, how the code implements it, how to use the website, and every term explained in plain words

Who should read this

This guide is written so that anyone on the team can read it once and then explain the whole project to the teacher: what problem it solves, why it does not use graph colouring, how each step works, where each step is in the code, what each tab of the website shows, and what the results mean. No earlier knowledge of the paper is assumed.

Team

Name	Reg. No.	Responsibility
Shreyans Chowdry	24BDS0114	Core pipeline, lexical and syntax analysis, AST, intermediate code
Samrender Singh Saini	24BCE2865	Semantic analysis, scope resolution, symbol table
Karthikeya Jami	24BCE2136	Optimisation engine, PGARA allocator, target code generation
Abhishek Kumar	24BCE0585	Runtime environment, test suite, benchmarking
Nishanth Kumar	24BAI0194	Web GUI, pipeline visualisation, documentation

Contents

1 PGARA on one page **3**

1.1 The problem . . . 3
1.2 Our idea in one sentence . . . 3
1.3 Why not graph colouring . . . 3
1.4 What we built . . . 3
1.5 The headline result . . . 3

2 Background in plain words **4**

2.1 What a compiler does . . . 4
2.2 Registers, memory and spilling . . . 5
2.3 How allocators usually decide . . . 5

3 The PGARA method, step by step **6**

3.1 Step 1: profiling (the “profile-guided” part) . . . 6
3.2 Step 2: how often each block runs . . . 6
3.3 Step 3: the Markov decision process of one value . . . 7
3.3.1 Bellman equations: the best you can do from here . . . 7
3.3.2 Policy iteration: how we solve it exactly . . . 7
3.4 Step 3b: the gain $G(v)$ . . . 8
3.5 Step 4: coalitions on φ-webs . . . 8
3.6 Step 5: the interference graph, chordal graphs and MCS . . . 9
3.7 Step 6: the sealed-bid auction . . . 9
3.8 Step 7: binding registers . . . 9
3.9 Step 8: best response, Nash equilibrium and coalition repair . . . 9
3.10 After allocation: back end and simulator . . . 10

4 Worked example: the rare error branch **10**

4.1 What a guess sees and what the profile sees . . . 11
4.2 The gains . . . 11
4.3 An auction round . . . 11
4.4 The outcome on the test input . . . 12

5 The research behind the project **12**

5.1 Foundations we build on . . . 12
5.2 The fifteen recent papers (2018–2024) and their gaps . . . 13
5.3 How our project relates to the original proposal . . . 14

6 How the code implements it **14**

6.1 The project at a glance . . . 14

6.2 From formula to function . . . 15

6.3 What happens when you press Compile . . . 16

6.4 How we know it is correct . . . 16

7 Using the website: PGARA Compiler Studio **17**

7.1 Starting it . . . 17

7.2 The top bar and the editor . . . 17

7.3 Tab by tab . . . 18

7.3.1 Tokens . . . 18

7.3.2 AST . . . 19

7.3.3 Symbols . . . 19

7.3.4 TAC . . . 20

7.3.5 SSA & Optimiser . . . 20

7.3.6 CFG . . . 21

7.3.7 Allocation (the most important tab) . . . 21

7.3.8 Target Code . . . 22

7.3.9 Run & Metrics . . . 23

7.3.10 Grammar . . . 23

7.4 A five-minute demo script . . . 24

8 The results and how to read them **24**

8.1 Setup . . . 24

8.2 Metrics . . . 24

8.3 Key numbers . . . 24

8.4 Honest limitations (say these before you are asked) . . . 25

8.5 Future work . . . 25

9 Glossary **25**

10 Likely viva questions, with answers **27**

11 Map of the research paper **29**

1 PGARA on one page

1.1 The problem

A processor has a handful of *registers*: tiny storage places inside the CPU that can be read in one cycle. Everything else lives in *memory*, which is several times slower. A program usually has far more variables than there are registers. So the compiler must decide, for every variable, whether it gets a register or has to live in memory. This decision is called **register allocation**, and putting a variable in memory is called **spilling** it. A bad choice makes the program slower, because every use of a spilled variable costs an extra memory load or store.

1.2 Our idea in one sentence

> **PGARA in one sentence**
>
> Run the program once on a sample input to **measure** which paths are actually taken (*profile-guided*); use those measured probabilities to compute, for every value, how many cycles it would save if it got a register (a small **Markov decision process**); then let the values **bid** those savings in an **auction** for the registers, and settle the result into a stable **Nash equilibrium**. When the measured behaviour changes, the allocation changes with it (*adaptive*).

1.3 Why not graph colouring

The classic textbook method (Chaitin's graph colouring) builds a graph whose nodes are variables and whose edges join variables that are alive at the same time, then tries to colour it with k colours (one colour per register). Our teacher asked us not to use it because the class already studies it. PGARA uses the same *interference graph* only as a map of who clashes with whom; it never tries to colour it. Instead the decisions come from **probability** (MDP) and **economics** (auction and game theory). That is the novelty.

1.4 What we built

- A complete compiler, written from scratch in Python, for **MiniC** (a small C subset): lexer, LL(1) parser, semantic analysis, three-address code, SSA form, optimiser, the PGARA allocator, code generation for our own small processor **PG-RISC**, and a cycle-counting simulator.
- Three other allocators to compare against: **Static-Prior PGARA** (same engine, but guessed probabilities instead of measured ones), **loop-depth priority** and **linear scan**.
- A colourful web interface, **PGARA Compiler Studio**, that shows every phase as you type.
- An IEEE-style research paper describing all of it.

1.5 The headline result



Geometric-mean cycles at $k = 4$ registers, relative to linear scan (lower is better)

On ten benchmark programs and five register budgets (200 compiled programs, all producing the correct output), at $k = 4$ PGARA needs **20.5% fewer cycles** than linear scan, 6.4% fewer than loop-depth priority and 2.5% fewer than Static-Prior, and does **59% fewer spill loads and stores** than linear scan. The gain is largest on programs whose branches depend on the data, which is exactly where measured probabilities beat guesses.

The 60-second pitch for the teacher

"Ma'am, register allocation decides which variables stay in the CPU's few registers. Usually it is done by graph colouring, which you asked us not to use. Our project, PGARA, treats it as a decision under uncertainty instead. We first run the program on a training input and count how often each branch is taken. For every variable we build a small two-state Markov decision process, register or memory, and solve it exactly with policy iteration. That gives the expected number of cycles the variable saves if it gets a register. Variables then bid that saving in a sealed-bid auction for the registers, and best-response dynamics settle the result into a Nash equilibrium. We built a full compiler around it with a web interface that shows every phase. On our benchmarks it is about 20% faster than linear scan and uses 59% fewer spill memory operations."

2 Background in plain words

2.1 What a compiler does

A compiler translates a program written by a person into instructions a machine can run. It does this in phases, each turning one representation into the next. Fig. 1 shows our phases; the purple ones are where PGARA lives.

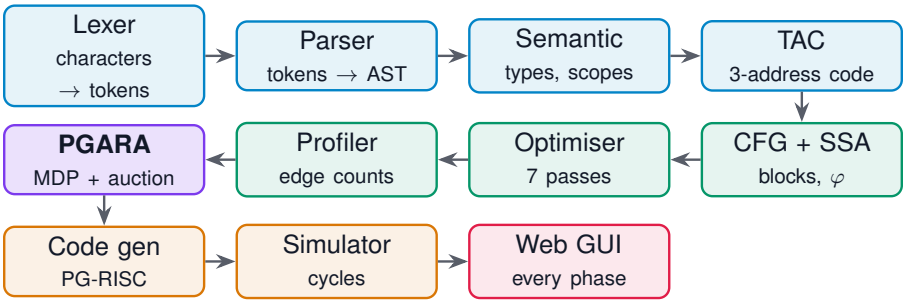


Figure 1: The phases of our compiler. Blue: front end. Green: middle end. Violet: the PGARA allocator. Amber: back end. Rose: the website.

Lexer (lexical analysis)
Reads characters and groups them into *tokens* such as `int`, `x`, `=`, `42`. Ours is a *maximal-munch DFA*: at each point it takes the longest piece of text that forms a valid token, so <= is one token, not < then =.

Parser (syntax analysis)
Checks that the tokens follow the grammar and builds an *abstract syntax tree* (AST). Our grammar is *LL(1)*: the parser can always choose the right rule by looking at just the next token. It has 41 nonterminals and 94 productions; the only conflict is the famous *dangling else*, resolved by attaching an `else` to the nearest `if`.

Semantic analysis
Checks meaning: is every variable declared, are types correct, does a function get the right number of arguments, is `break` inside a loop. It builds the *symbol table* with one scope per block and gives each variable a unique name (a second variable called `i` in the same function becomes `i#1`), so two different variables are never confused.

Three-address code (TAC)
A simple intermediate language where each instruction does one operation, like `t3 = i % 3`. Conditions with && and || become *jumping code*, so short-circuit evaluation is exact.

CFG and SSA
The code is cut into *basic blocks* (straight runs with no jumps in the middle) joined by edges: the *control-flow graph*. Then it is put into *static single assignment* form, where every variable is assigned exactly once. Where two paths meet, a φ-function picks the right version (for example `i.2 =` φ`(0, i.3)` at the top of a loop).

Optimiser
Makes the code smaller and faster without changing its output: constant propagation (SCCP), copy propagation, algebraic simplification, global value numbering (removes repeated calculations), dead code elimination, block merging and compare-branch fusion.

Profiler
Runs the optimised program on the *training input* and counts how many times each CFG edge was taken.

Register allocation
Decides which values get registers. This is our research contribution.

Code generation
Writes PG-RISC assembly: replaces φs by copies, inserts loads and stores for spilled values, saves registers around calls.

Simulator
Runs the assembly on the *test input*, counts cycles and checks that the output equals the reference interpreter's output.

2.2 Registers, memory and spilling

> **Everyday picture: the desk and the cupboard**
>
> Think of a student with a small desk that holds only four books (the registers) and a cupboard across the room that holds everything (memory). Fetching a book from the cupboard takes time. If you are going to read one book a hundred times tonight and another only once, the first one should stay on the desk. Register allocation is choosing which books stay on the desk. The hard part is that you do not always know in advance how often you will need each book. PGARA watches you work for one evening (the profile) and then decides.

In PG-RISC a register access is free, a *load* from memory costs 4 cycles and a *store* to memory costs 2 cycles. So a spilled variable that is used in a loop running a million times can cost millions of cycles.

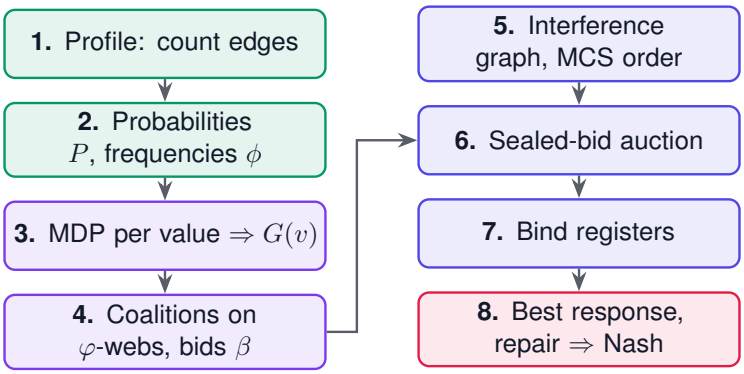
2.3 How allocators usually decide

- **Graph colouring** (Chaitin 1982): colour the interference graph with k colours; if impossible, spill the node with the lowest *cost / degree*. The cost is a static guess, usually $10^{\text{loop depth}}$.
- **Linear scan** (Poletto and Sarkar 1999): sort the variables by where their lifetime starts, walk through them once, and when registers run out spill the one whose lifetime ends last. Very fast, but blind to how often code runs.
- **Priority by loop depth** (Chow and Hennessy 1990): a use inside one loop counts 10, inside two loops 100, and so on.

All of them *guess* how often code runs. A loop that contains an `if` whose `then` branch is an error case taken 1% of the time looks, to them, like a 50/50 split. PGARA *measures* it.

3 The PGARA method, step by step

This section explains every idea in the order the allocator uses them. Each part has a plain explanation, the formula from the paper, and where to find it in the code. The worked example in Section 4 puts real numbers on all of it.



3.1 Step 1: profiling (the "profile-guided" part)

The compiler first runs the optimised program in its own *reference interpreter* on a **training input**, and counts every time control goes from block b to block s. Call that count $n(b, s)$. Dividing by the total number of times b was left gives the probability of each edge:

$$P_{\text{prof}}(b, s) = \frac{n(b, s)}{\sum_{s'} n(b, s')}.$$

Later the program is measured on a **different test input**. Training and testing on different inputs is important: it shows the profile generalises and we are not cheating.

Profiled versus static probabilities

If a block was never reached in training, or for the *Static-Prior* variant, we fall back to a classic guess: inside a loop the edge that stays in the loop gets 0.9 and the exit 0.1; any other branch is split evenly (0.5 / 0.5 for an `if`). The CFG tab of the website shows both numbers on every edge: *profiled (static)*.

Where it is in the code

`interp.py`: `Interpreter.call` counts `edge_counts`. `regalloc.py`: `profiled_probabilities`, `static_probabilities`.

3.2 Step 2: how often each block runs

Knowing the edge probabilities, we can compute how many times each block runs, on average, per call of the function. Call it $\phi(b)$. The entry block runs once; every other block runs as often as control flows into it:

$$\phi = e_{\text{entry}} + P^{\top}\phi.$$

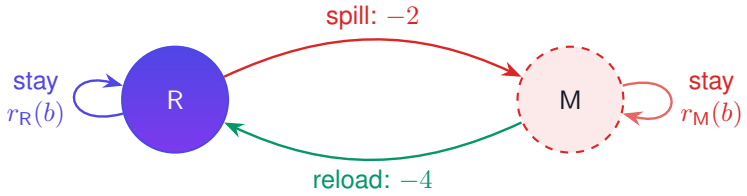
This is a small linear system, solved exactly by Gaussian elimination. In a loop that runs 10 times, the loop body gets $\phi \approx 10$.

> **Where it is in the code**
> `regalloc.py`: `block_frequencies`, using `solve_linear`.

3.3 Step 3: the Markov decision process of one value

This is the heart of PGARA. Take one SSA value v, for example the loop counter `i.2`. Its *live range* is the set of blocks where it is still needed. We model its life as a small game against chance:

- **States.** At each block b of the live range the value is either in a register (R) or in memory (M).
- **Actions.** In R you may *stay* or *spill* (costs a store, 2 cycles). In M you may *stay* or *reload* (costs a load, 4 cycles).
- **Chance.** Control moves to the next block with the measured probability $P(b, b')$. Leaving the live range ends the game.
- **Rewards.** In R, every use or definition in the block saves a cycle. In M, every use costs a load (4) and every definition a store (2).



The precise rewards in block b, with u uses, d definitions, f *forwardable* uses (a use right after the definition, which becomes a cheap move) and c calls crossed, are

$$r_{\mathsf{R}}(b) = 1 \cdot (u + d + f) - c\,(2 + 4), \qquad r_{\mathsf{M}}(b) = -(4u + 2d + 1f).$$

A register costs something too: across a call it must be saved and restored.

Bellman equations: the best you can do from here

$V(b, s)$ means "the best total reward you can still collect if you are in block b in state s". Bellman's principle says: the best from here is today's reward plus the best choice for tomorrow, averaged over where control goes:

$$V(b, \mathsf{R}) = r_{\mathsf{R}}(b) + \max\{\bar{V}_{\mathsf{R}}(b),\ -2 + \bar{V}_{\mathsf{M}}(b)\},$$
$$V(b, \mathsf{M}) = r_{\mathsf{M}}(b) + \max\{\bar{V}_{\mathsf{M}}(b),\ -4 + \bar{V}_{\mathsf{R}}(b)\}, \qquad \bar{V}_s(b) = \textstyle\sum_{b'} P(b, b')\, V(b', s).$$

Policy iteration: how we solve it exactly

A *policy* says what to do in every state (stay or switch). Howard's **policy iteration** solves the MDP exactly:

1. Start with "always stay".
2. **Evaluate** the policy: its values satisfy a linear system $(I - P_\pi)V_\pi = r_\pi$; solve it.
3. **Improve**: in every state pick the action that is best according to those values.
4. If nothing changed, the policy is optimal; otherwise go to step 2.

It finishes in very few rounds (on our benchmarks 1.0 to 1.5 rounds on average, at most 7). The website's Allocation tab shows this as *PI rounds*. We also implemented *value iteration* and a unit test checks both give the same answer.

Why no discount factor

Many MDPs multiply future rewards by a discount $\gamma < 1$. We do not need one: a program always leaves a live range eventually, so the total reward is finite. This is called a *total-reward* or *transient* MDP. Only if that ever failed would the code fall back to $\gamma = 0.999$.

Where it is in the code

`regalloc.py`: class `ValueMDP` with `q` (one Bellman step), `evaluate` (linear solve), `policy_iteration`, `value_iteration` (cross-check), `memory_only` and `solve`.

3.4 Step 3b: the gain $G(v)$

The **gain** of a value is the number we auction on. It answers: "how many cycles, per call of the function, do we save by giving v a register instead of leaving it in memory?"

$$G(v) = \phi\big(def(v)\big) \cdot \Big(V^*\big(def(v), \mathsf{R}\big) - V_{\text{mem}}\big(def(v)\big)\Big).$$

In words: (how often the value is created) $\times$ (best reward with a register $-$ reward if it stays in memory the whole time).

A tiny example you can check by hand

`int x = input(); print(x); print(x + 1);` The value `x` is defined once and used twice, in a block that runs once.

- The definition: in a register it saves 1; in memory it costs a store, 2. Difference $1 + 2 = 3$.
- The first use comes right after the definition, so in memory it is a forwarded move costing 1; in a register it saves 1. Difference $1 + 1 = 2$.
- The second use: register saves 1, memory costs a load of 4. Difference $1 + 4 = 5$.

$G(\mathtt{x}) = 3 + 2 + 5 = 10$ cycles. The unit test `test_hand_computed_single_block_gain` checks that the code gives exactly this.

3.5 Step 4: coalitions on φ-webs

In SSA form a loop variable like `i` becomes several values: `i.2` at the loop top, `i.3` after `i++`, joined by a φ. If `i.2` got a register but `i.3` did not, every trip round the loop would copy through memory, which is wasteful. So values joined by φ over a *hot* edge form a **coalition** (a team). An edge is hot when it carries at least a quarter ($\theta = 0.25$) of the traffic of its source or target:

$$\phi(b)\,P(b,h) \;\geq\; 0.25 \cdot \max\{\phi(def(a)),\ \phi(h)\}.$$

Every member of a coalition bids the **sum of the team's gains**, $\beta(v) = \sum_{w \in C(v)} G(w)$, so the team wins or loses together. This is the idea of *transferable utility* from cooperative game theory (Shapley).

Where it is in the code

`regalloc.py`: `phi_webs`, `coalitions` (union-find, $\theta = 0.25$), `coalition_bids`.

3.6 Step 5: the interference graph, chordal graphs and MCS

Two values *interfere* if they are alive at the same time; then they cannot share a register. Drawing a node per value and an edge per clash gives the **interference graph**. We use it as a map, not to colour.

A useful fact (Hack et al. 2006): for programs in SSA form, the interference graph is always **chordal**, meaning every cycle of four or more nodes has a shortcut edge. Chordal graphs have a special ordering, found by **maximum cardinality search (MCS)**: repeatedly visit the node with the most already-visited neighbours. In that order, when a node arrives, its already-visited neighbours all clash with each other: they form a **clique**, a group of values that are all alive at the same moment. The reverse of this order is a **perfect elimination ordering (PEO)**, and the code checks it on every function.

> **Why the clique matters**
>
> A clique of size 6 with $k = 4$ registers means: at some moment six values are alive and only four desks are free. Exactly two must go to the cupboard. The clique is the precise place where competition happens, so that is where we hold the auction.

> **Where it is in the code**
>
> `regalloc.py`: `FunctionAnalysis.build_interference`, `maximum_cardinality_search`, `verify_peo`, `chordal_structure`.

3.7 Step 6: the sealed-bid auction

Values are visited in MCS order. If a value has fewer than k neighbours that already hold registers, it simply wins. Otherwise the clique is over budget, and an **auction round** is held among the clique: every member submits its bid β (sealed: no one adapts to others' bids), and the k highest win. Ties are broken by MCS order, so the result is always the same. The losers are spilled.

$$W(Q) = \arg \max_{S \subseteq Q,\ |S| \le k} \sum_{w \in S} \beta(w) \quad \text{(solved exactly by sorting).}$$

3.8 Step 7: binding registers

Winners then get actual register numbers, again in MCS order: each takes a register not used by any winning neighbour, preferring the one a team-mate in its φ-web already holds (so the copy disappears). Because the auction never lets more than k winners clash, a free register always exists.

> **Where it is in the code**
>
> `regalloc.py`: `auction`, `bind_registers`.

3.9 Step 8: best response, Nash equilibrium and coalition repair

The auction decides each clique locally, so sometimes a loser ends up with a register that is actually free (because the rival that beat it later lost somewhere else). We fix this with **best-response dynamics**: any spilled coalition whose members can all find a free register takes them, highest bid first, repeated until nobody can improve. At that point no player can gain by changing its own choice alone: this is a **pure Nash equilibrium**.

Coalition repair handles the opposite case. If a team ended up split (some members in registers, some in memory), every hot φ edge between them costs a copy through memory. If that expected copy cost is at least what the members in registers gain, the whole team *concedes* and gives up its registers, and best

response runs again so others can use them. The website shows both counts as *best-response moves* and *coalition concessions*.

Why it is called a game

"Each coalition is a player. Its strategy is a register for each member, or memory. Its payoff is its bid if all its members get registers, otherwise zero. Registers are the shared resource, which makes it a congestion game (Rosenthal). Best response stops after at most $|V|$ moves because each move only adds values to registers, and the final state is a Nash equilibrium."

Where it is in the code

`regalloc.py`: `best_response`, `repair_coalitions`; everything is called in order by `allocate_function`.

3.10 After allocation: back end and simulator

- **Out of SSA.** φ-functions become *parallel copies* on the incoming edges. Cycles of copies (like swapping two registers) are broken through a scratch register `t2`.
- **Spill slots.** A spilled value lives in a slot in the stack frame. All members of one φ-web share one slot, so spilled φ copies cost nothing.
- **Calls.** Registers alive across a call are saved before it and restored after it (caller-save).
- **Peephole.** Small clean-ups: remove jumps to the next line, thread jumps to jumps, and turn "store then immediately load the same slot" into a register move (*store-to-load forwarding*).
- **Simulator.** Runs the PG-RISC code with fixed latencies (load 4, store 2, ALU 1, multiply 3, divide 12, taken branch 2, not-taken 1, call and return 3), counts cycles, and checks the output against the interpreter.

Where it is in the code

`codegen.py`: `generate_function`, `parallel_copy`, `peephole`. `simulator.py`: `Simulator.run`.

4 Worked example: the rare error branch

Benchmark 5 (*Rare error branch*) is the best program to demonstrate PGARA. Its inner loop reads a number `x`; if `x` is negative (an "error", which happens about 1% of the time) it updates four error counters, otherwise it updates four normal counters:

```
for (int i = 0; i < n; i++) {
  int x = data[i];
  if (x < 0) {            // rare: about 1% of iterations
    errCount++; errSum += x;
    if (lastErr >= 0) errSpan += i - lastErr;
    lastErr = i;
  } else {                // common: about 99%
    sum += x * 2; weighted += x * (i % 7); cnt++;
    if (x > peak) peak = x;
  }
}
```

The training input is `7 10` (seed 7, error threshold 10 per 1000) and the test input is `13 10` (a different seed). With $k = 4$ registers there are far more live values than registers, so something must be spilled.

4.1 What a guess sees and what the profile sees

Edge	Static guess	Measured (profile)
loop continues	0.9	0.999
`x < 0` (error path)	0.5	**0.01**
`x >= 0` (normal path)	0.5	**0.99**

The static guess treats the two branches as equally likely; the profile knows the error path is 99 times rarer.

4.2 The gains

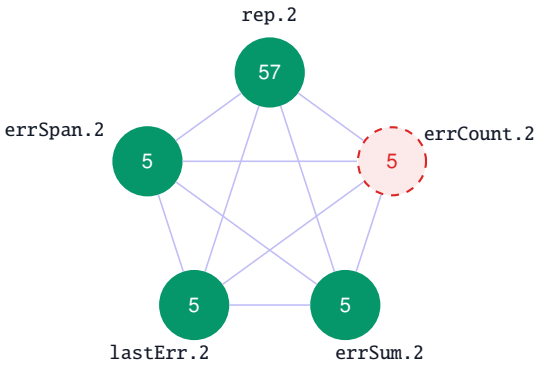
Value	G with static guess	G with profile	Meaning
`cnt.3`, `weighted.3`	202.5	**20,300**	normal-path counters
`errSum.3`, `errCount.3`	202.5	**180**	error-path counters
`x`		101,860	loaded every iteration
`i#1.2`		81,935 (bid 94,223)	loop counter at loop top
`i#1.3`		12,288 (bid 94,223)	loop counter after `i++`

Things to notice:

- Under the static guess all four counters look identical (202.5), so the allocator cannot tell them apart. Static-Prior actually keeps the error-path value `lastErr` in a register and spills `cnt` and `weighted`: the wrong way round.
- With the profile, the normal-path counters are worth $20{,}300/180 \approx 113$ times more, and PGARA keeps them in registers.
- `i#1.3` on its own is worth only 12,288, but it is in a coalition with `i#1.2`, so both bid the team total $81{,}935 + 12{,}288 = 94{,}223$ and they share register `r1`. That is the coalition idea working.

4.3 An auction round

When `errCount.2` arrives in MCS order, its clique has five members but there are only four registers:



The numbers are bids. The four highest win (green). Four values tie at 5, and the tie is broken by MCS order, so `errCount.2` loses (red, dashed). This round is in the outer part of the program where these values are cheap, so the choice hardly matters, which is exactly what the low bids say.

4.4 The outcome on the test input

Allocator	Cycles	Spill loads + stores	Register utilisation
PGARA (profiled)	**338,389**	**28,835**	**78.7%**
Static-Prior PGARA	390,709	44,888	70.4%
Loop-depth priority	423,175	52,994	71.2%
Linear scan	516,434	82,911	55.7%

PGARA is 34% faster than linear scan and 13% faster than the same engine without the profile. All four produce the same, correct output.

5 The research behind the project

5.1 Foundations we build on

These are the classic works whose ideas PGARA combines. You should be able to say in one line what each one gave us.

Work	What we took from it
Bellman (1957)	The principle of optimality and the Bellman equations used in the MDP.
Howard (1960)	Policy iteration, the exact solver we use for each value's MDP.
Puterman (1994); Bertsekas (2019); Sutton & Barto (2018)	Theory of Markov decision processes, total-reward (undiscounted) MDPs, value iteration.
Vickrey (1961)	Sealed-bid auctions: each bidder submits one bid without seeing the others.
Shapley (1953)	Cooperative games with transferable utility: the idea that a coalition's members share the coalition's total value (our φ-web bids).
Rosenthal (1973); Nisan et al. (2007)	Congestion games and pure Nash equilibria; best-response dynamics.
Tarjan & Yannakakis (1984)	Maximum cardinality search and the test for chordal graphs.
Hack, Grund & Goos (2006)	SSA programs have chordal interference graphs, so cliques are exactly the points of register pressure.
Cytron et al. (1991)	Construction of SSA form with φ-functions using dominance frontiers.
Cooper, Harvey & Kennedy (2001)	The simple, fast dominator algorithm we use.
Wegman & Zadeck (1991)	Sparse conditional constant propagation (SCCP) in our optimiser.
Wu & Larus (1994); Wagner et al. (1994)	Static branch-probability heuristics, used by Static-Prior, and the case for profile feedback.

Work	What we took from it
Chaitin (1982)	Graph-colouring allocation: the method we were asked *not* to use.
Chow & Hennessy (1990)	Priority-based allocation with loop-depth weights: one of our baselines.
Poletto & Sarkar (1999)	Linear-scan allocation: our main baseline.
Knuth (1965, 1968); Lewis & Stearns (1968)	LR/LL parsing theory and attribute grammars behind our front end.
Aho, Lam, Sethi & Ullman (2006); Cooper & Torczon (2022)	The textbooks ("Dragon book" and "Engineering a Compiler") for the front end, TAC and jumping code.

5.2 The fifteen recent papers (2018–2024) and their gaps

The paper's Related Works section reviews fifteen recent works. For each one, the important thing to remember is its **gap**: what it does not do that PGARA does.

Paper	What they did	Gap that PGARA fills
Combinatorial and survey work on allocation		
Castañeda Lozano et al. 2019, *Unison*	Register allocation and scheduling as one constraint-programming model inside LLVM; optimal for medium functions.	Optimises a *static* cost estimate; solve time grows fast with size.
Castañeda Lozano & Schulte 2019, survey	Surveys integer, constraint and PBQP models for allocation and scheduling.	Almost all models assume static frequencies, not measured stochastic behaviour.
Learning-based allocation and program representation		
VenkataKeerthy et al. 2023, *RL4ReAl*	Reinforcement-learning register allocator in LLVM, hierarchy of agents.	Costly training; decisions are not explainable as expected savings.
Trofin et al. 2021, *MLGO*	Trained policies inside LLVM for inlining and eviction.	Learns from a corpus, not from the profile of the program being compiled.
Cummins et al. 2022, *CompilerGym*	Gym environments for compiler optimisation research.	Targets pass-level choices, not per-value register decisions.
Cummins et al. 2021, *ProGraML*	Graph neural networks over control, data and call flow.	Learns to approximate analyses that PGARA computes exactly.
VenkataKeerthy et al. 2020, *IR2Vec*	Vector embeddings of LLVM IR.	Captures syntax and flow, not execution frequencies.
Jain et al. 2022, *POSET-RL*	RL to order optimisation passes.	Treats the allocator as a black box.
Haj-Ali et al. 2020, *NeuroVectorizer*	Deep RL to choose vectorisation factors.	Rewards are measured, but the decision has no model of program behaviour.
Surveys of machine learning in compilers		
Wang & O'Boyle 2018	Survey of ML in compiler optimisation.	Register allocation barely covered; role of measured profiles left open.

Paper	What they did	Gap that PGARA fills
Ashouri et al. 2018	Survey of autotuning and phase ordering.	Focus on choosing passes, not on assigning resources inside a pass.
Profile-guided optimisation		
Panchenko et al. 2019, *BOLT*	Post-link optimiser using sampled hardware profiles to reorder code.	Profiles used only for layout; registers are already fixed.
Shen et al. 2023, *Propeller*	Profile-guided relinking for very large applications.	Same as BOLT: the profile never reaches allocation.
Rotem & Cummins 2021	Predicts branch probabilities with ML so PGO works without profiles.	Feeds existing static heuristics, not an allocation model.
Infrastructure and large language models		
Lattner et al. 2021, *MLIR*	Multi-level compiler infrastructure.	Lowers to LLVM's allocators; does not change how registers are assigned.
Cummins et al. 2023, LLM optimiser	A 7B language model that predicts optimisation passes.	Outputs not guaranteed correct; no account of decisions.

The research gap in two sentences

"Measured execution frequencies are used today for code layout (BOLT, Propeller) but not for register allocation, learned allocators need heavy training and cannot explain their choices, and combinatorial allocators optimise static cost models. PGARA sits in that gap: an allocator whose objective is the expected saving under measured probabilities, solved exactly in milliseconds, with every decision explainable as a bid."

5.3 How our project relates to the original proposal

Our earlier proposal (Review 1) described the same method. Two things changed while building it, and both are stated in the paper:

- The proposal planned value iteration; the final code uses **policy iteration**, which is exact and needs only 1 to 7 rounds. Value iteration is kept as a cross-check in the tests.
- The proposal planned to compare against LLVM's allocator. Our target PG-RISC is not an LLVM target, so for a fair comparison we compare against **loop-depth priority** and **linear scan**, running on exactly the same compiler, optimiser and simulator.

6 How the code implements it

6.1 The project at a glance

The whole compiler is plain Python 3 with no extra packages, about 6,000 lines including the website. It is organised into one Python package, `pgara`, plus the server, the web pages, the benchmarks and the tests.

File	What it does	Owner
`lexer.py`	Maximal-munch DFA lexer with line/column positions and error recovery.	Shreyans

File	What it does	Owner
`grammar.py`	The LL(1) grammar, FIRST/FOLLOW sets and predictive-table check.	Shreyans
`parser.py`	Recursive-descent parser, one method per nonterminal, panic-mode recovery.	Shreyans
`ast.py`	AST node class used by every later phase.	Shreyans
`semantic.py`	Symbol table, scopes, type checking, unique IR names, diagnostics.	Samrender
`ir.py`	The three-address-code data structures (instruction, block, function, module).	Shreyans
`irgen.py`	Translates the AST into TAC, with jumping code for &&, ||, !.	Shreyans
`cfg.py`	Basic blocks, critical-edge splitting, dominators, natural loops, loop depth.	Karthikeya
`ssa.py`	Liveness analysis and pruned SSA construction with φ-functions.	Karthikeya
`optimize.py`	SCCP, copy propagation, simplification, GVN, DCE, block merge, branch fusion.	Karthikeya
`interp.py`	Reference interpreter: the run-time environment, and the edge profiler.	Abhishek
`regalloc.py`	**The PGARA allocator** (MDP, gains, coalitions, MCS, auction, best response) and the three baselines.	Karthikeya
`codegen.py`	PG-RISC code generation, out-of-SSA copies, spill slots, peephole.	Karthikeya
`simulator.py`	Cycle-counting PG-RISC simulator.	Abhishek
`pipeline.py`	Runs every phase in order and builds the report the website shows.	Shreyans
`bench.py`	Runs the whole benchmark study and writes `results/`.	Abhishek
`server.py`	Small web server: serves the pages and answers `/api/compile`.	Nishanth
`web/`	`index.html`, `app.css`, `app.js`: the PGARA Compiler Studio.	Nishanth
`tests/`	23 unit tests covering every phase.	Abhishek
`benchmarks/`	The ten MiniC benchmark programs.	Abhishek

6.2 From formula to function

Every formula in the paper has one place in the code. This table is useful if the teacher asks “show me where you do this”. All functions are in `pgara/regalloc.py` unless another file is named.

Idea	Formula	Function
Edge counting	$n(b, s)$	`interp.py: Interpreter.call`
Profiled probabilities	$P_{\text{prof}} = n(b, s)/\sum n$	`profiled_probabilities`
Static guesses	0.9 / 0.1, or even split	`static_probabilities`
Block frequencies	$\phi = e + P^{\top}\phi$	`block_frequencies`
Use/def/call counts	u, d, f, c	`FunctionAnalysis.count_accesses`
Rewards and Bellman step	$r_{\text{R}}, r_{\text{M}}, V(b, s)$	`ValueMDP.__init__, ValueMDP.q`
Policy evaluation	$(I - P_{\pi})V_{\pi} = r_{\pi}$	`ValueMDP.evaluate, solve_linear`
Policy iteration	improve until stable	`ValueMDP.policy_iteration`
Memory-only value	V_{mem}	`ValueMDP.memory_only`
Gain	$G(v)$	`ValueMDP.solve, compute_gains`
Hot-edge coalitions	$\theta = 0.25$	`coalitions, phi_webs`
Coalition bid	$\beta(v) = \sum G(w)$	`coalition_bids`
MCS order and PEO check		`maximum_cardinality_search, verify_peo`
Sealed-bid auction	top-k of the clique	`auction`
Register binding	lowest free / partner's register	`bind_registers`
Best response (Nash)	no profitable deviation	`best_response`
Coalition repair	split cost $\geq$ winners' gain	`repair_coalitions`
Loop-depth baseline	10^{depth}	`loop_depth_weights`
Linear-scan baseline	intervals	`linear_scan`
Everything in order		`allocate_function`

6.3 What happens when you press Compile

1. The browser (`web/app.js`) sends the source code, k, the optimisation level and the two inputs to `server.py` at `/api/compile`.
2. `server.py` calls `compile_source` in `pipeline.py`.
3. `compile_source` runs: lexer → parser → semantic analysis (stopping here with diagnostics if there are errors) → TAC → CFG and SSA → optimiser → profile on the training input → all four allocators → code generation → simulation on the test input.
4. It returns one JSON report with the result of every phase.
5. `app.js` draws every tab from that report: the token list, the AST, the CFG, the interference graph, the assembly and the charts.

6.4 How we know it is correct

- **23 unit tests** (`python3 -m unittest -v tests.test_compiler`): maximal munch, error recovery, the grammar is LL(1) except dangling else, precedence, scopes, SSA single-assignment and dominance, optimiser preserves output, MCS on chordal and non-chordal graphs, every SSA interference graph is chordal, policy iteration equals value iteration, the hand-computed gain of 10, hot values outbid cold ones by more than 10 times, no two clashing values share a register, the Nash property, and every allocator reproduces the reference output.
- **Every simulated run is checked** against the reference interpreter. In the full study all 200 runs match.

7 Using the website: PGARA Compiler Studio

7.1 Starting it

Unzip `PGARA-CompilerProject.zip`. Then:

macOS	double-click `start.command` (first time: right-click, Open, Open)
Windows	double-click `start.bat`
Any terminal	`python3 server.py`

The browser opens `http://127.0.0.1:8765/` by itself. Only Python 3.8 or newer is needed. Press Ctrl+C in the terminal to stop.

7.2 The top bar and the editor

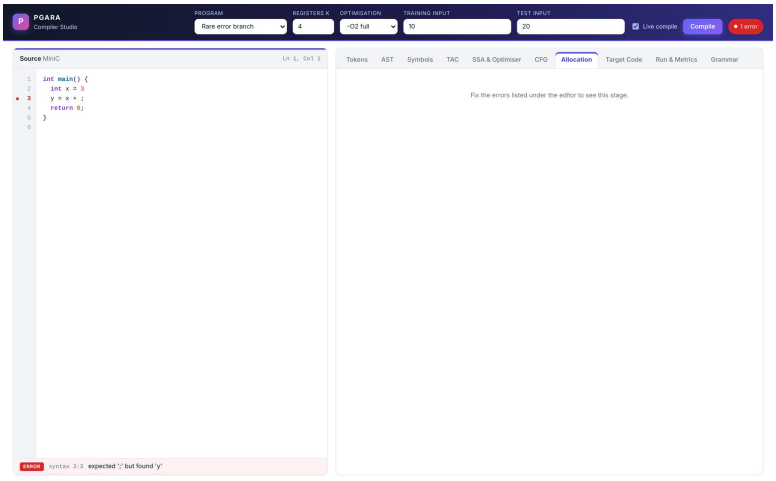
Top bar



- **Program**: pick one of the ten benchmarks, or type your own code.
- **Registers k**: how many registers the allocator may use (2 to 16). Try lowering it to see more spilling.
- **Optimisation**: the optimiser level (`-O2 full` by default).
- **Training input**: numbers read by `input()` during *profiling*. **Test input**: numbers used for the *measured* run. Use different numbers to show the profile generalises.
- **Live compile**: recompiles as you type. The green pill shows the compile time.

The editor on the left colours the code (keywords purple, numbers orange, comments green, operators pink, function names blue). Errors show in red in the gutter and in a list below; clicking an error jumps to its line.

An example with errors: a missing semicolon and a broken expression

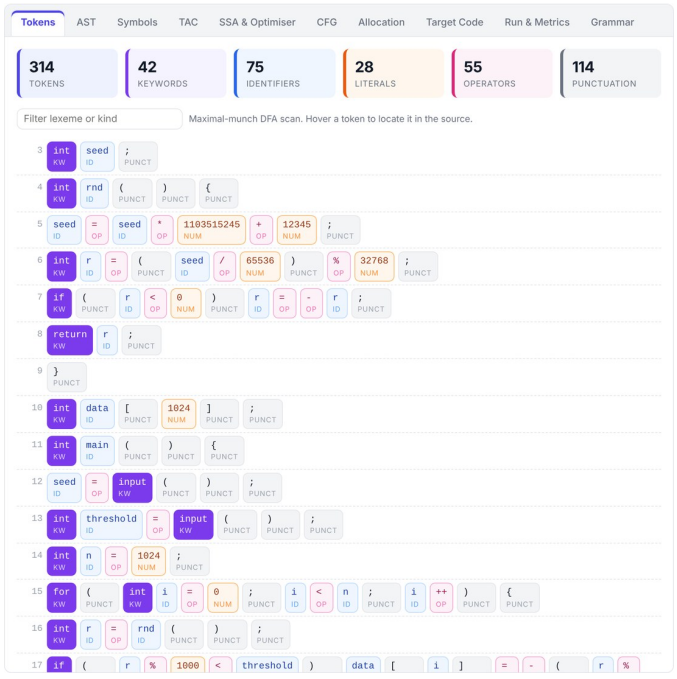


7.3 Tab by tab

For the screenshots below, the program is the rare-branch benchmark at $k = 4$, except where a smaller loop is used to keep the picture readable.

Tokens

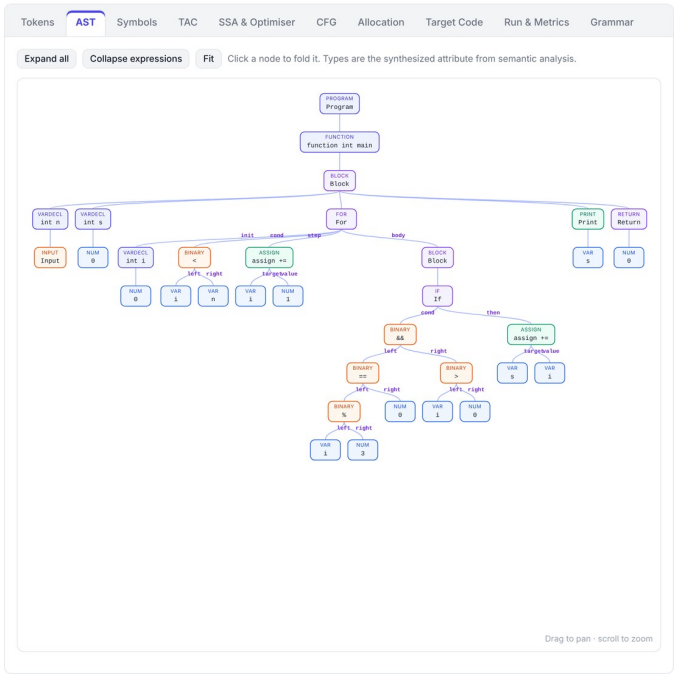
Tokens tab



The lexer's output: each token with its kind and position, coloured by kind. Hover a token to highlight it in the source. **Point out:** maximal munch (<=, ++, += are single tokens).

AST

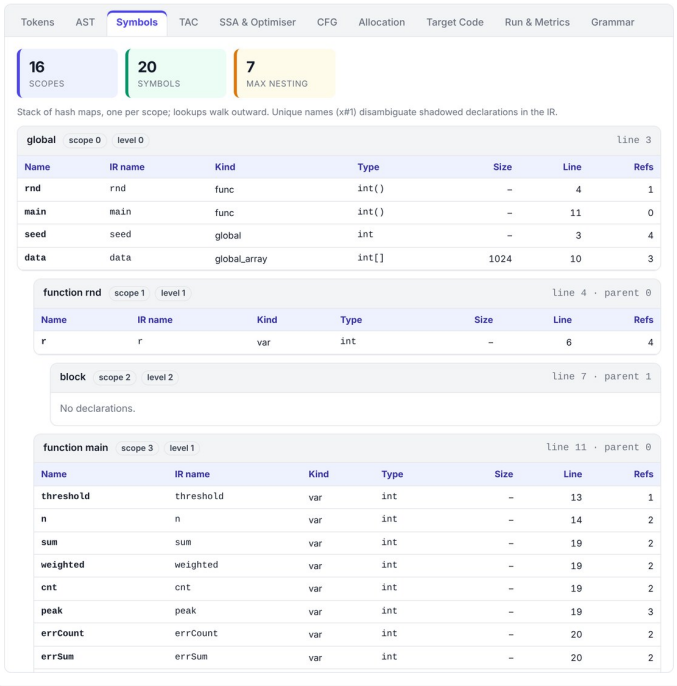
AST tab (small loop program)



The syntax tree. Drag to move, scroll to zoom, click a node to fold it. Colours: declarations indigo, control flow violet, statements green, expressions orange, leaves blue. **Point out:** operator precedence is visible in the shape of the tree: operators that bind tighter sit lower (== below &&).

Symbols

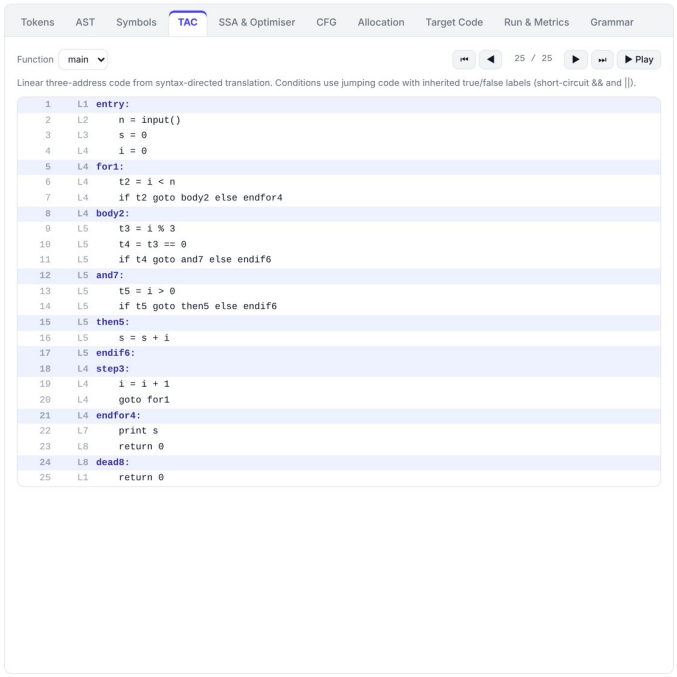
Symbols tab



One card per scope with every symbol's unique IR name, kind, type and how many times it is referenced. **Point out:** two variables named `i` in different loops get different names (`i` and `i#1`).

TAC

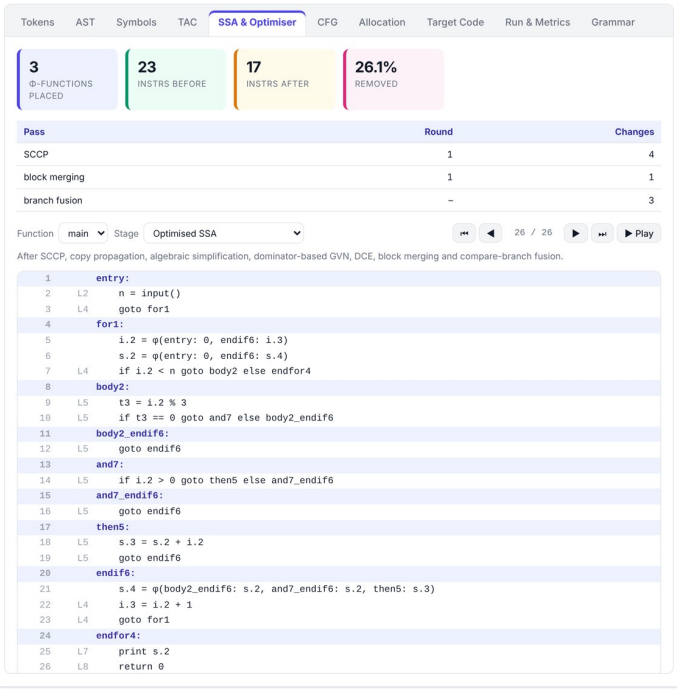
TAC tab (small loop program)



Three-address code. Use the step buttons or Play to watch it being generated one instruction at a time. **Point out:** && becomes two separate jumps (jumping code).

SSA & Optimiser

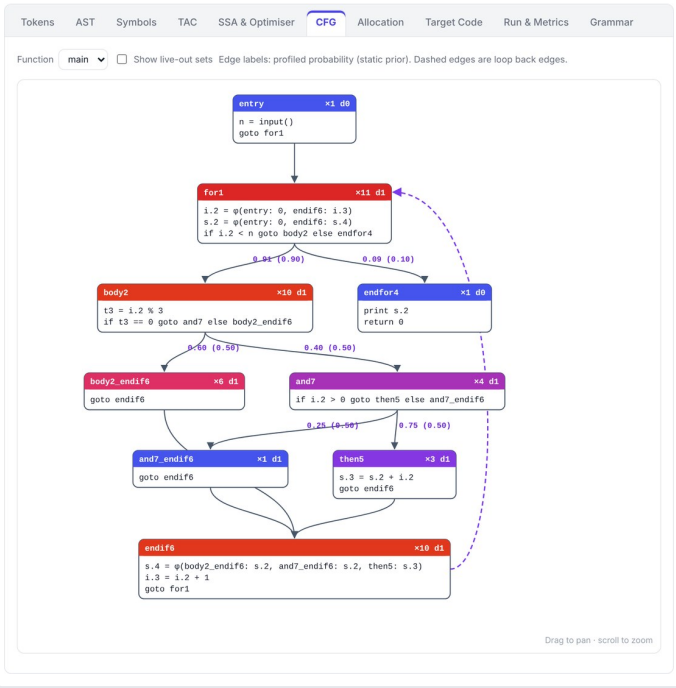
SSA & Optimiser tab (small loop program)



The program in SSA form with φ-functions, then after optimisation, with a log of what each optimiser pass changed. **Point out:** `i.2 = φ(0, i.3)` at the loop head; the log shows constants folded and dead code removed.

CFG

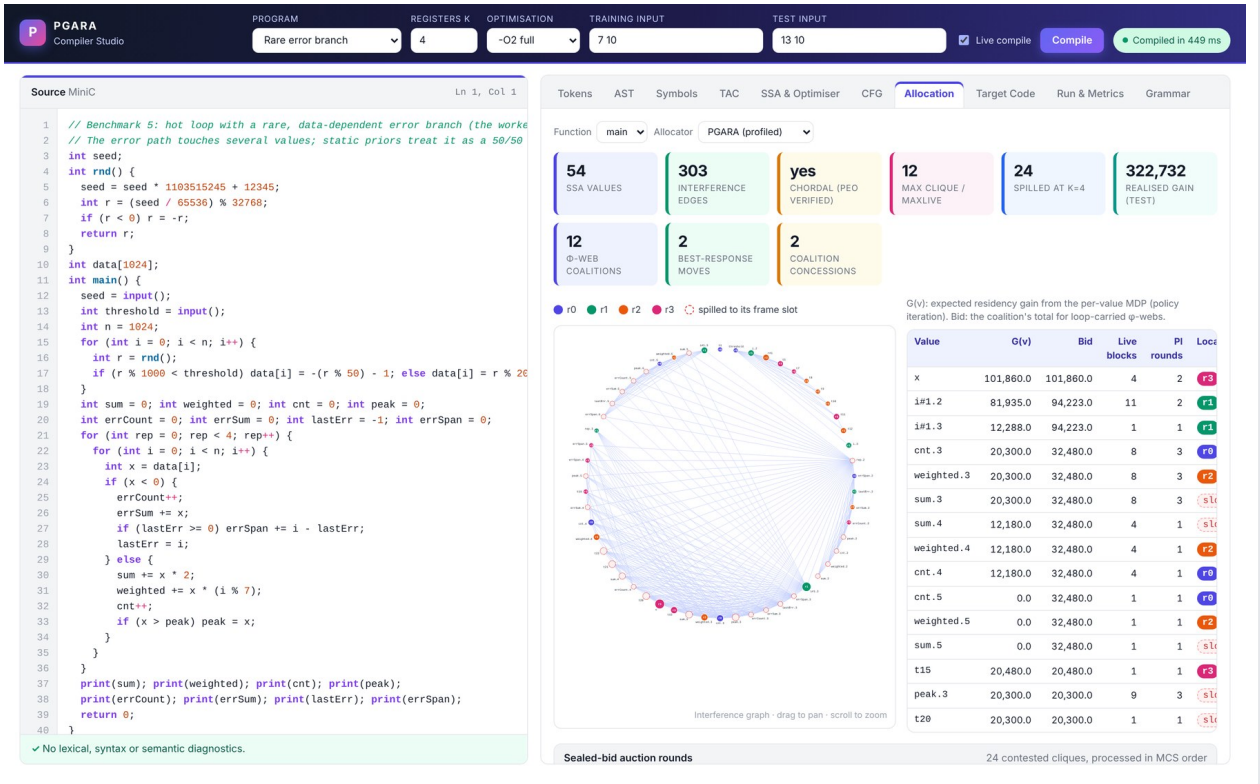
CFG tab (small loop program)



The control-flow graph. Each block header shows how often it ran in training (×11) and its loop depth (d1); the colour goes from blue (rare) to red (hot). Each edge shows *profiled probability (static guess)*. The violet dashed edge is the loop's back edge. **Point out:** the edge `0.25 (0.50)`: the guess says 50%, the profile says 25%. This difference is what PGARA exploits.

Allocation (the most important tab)

Allocation tab: rare-branch benchmark, $k = 4$



- **Cards at the top**: number of SSA values, interference edges, "chordal (PEO verified): yes", the

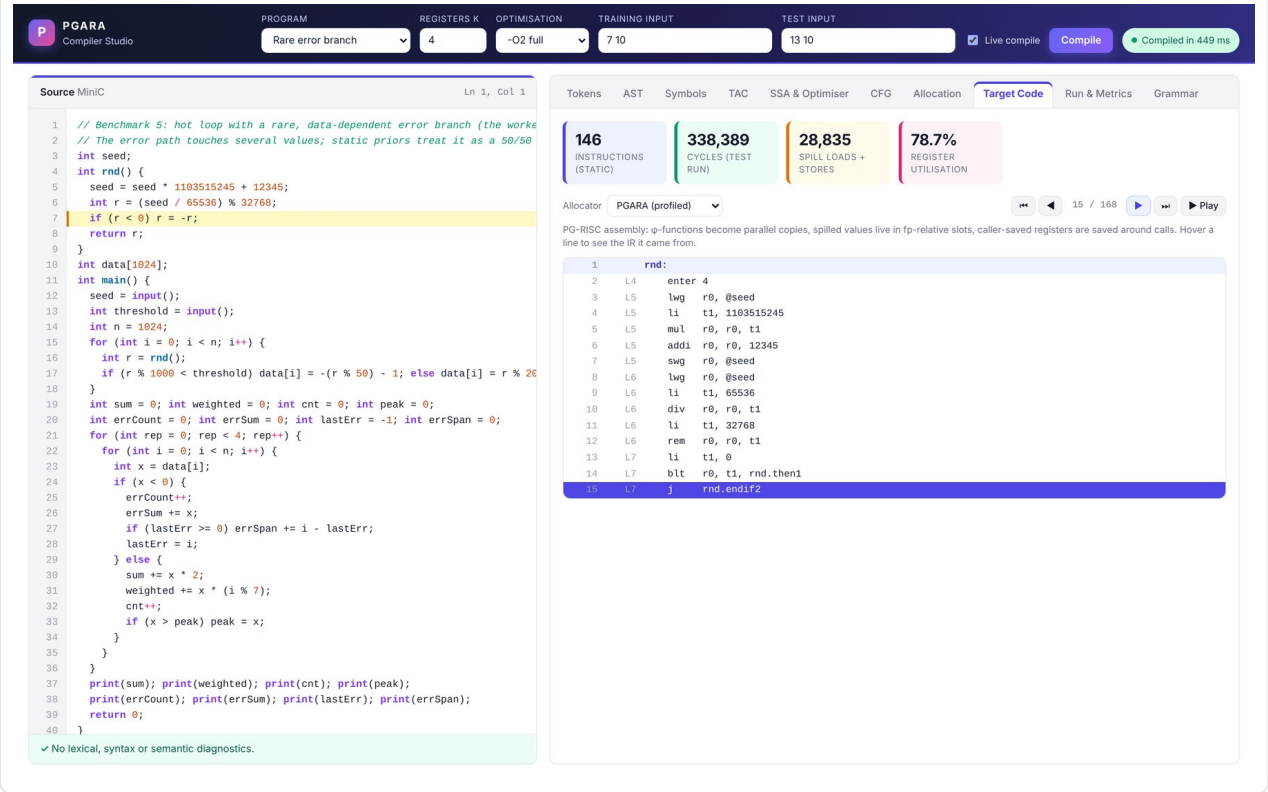
largest clique, how many values were spilled, the realised gain on the test input, the number of φ-web coalitions, best-response moves and coalition concessions.

- **Interference graph**: each node is a value, coloured by its register (`r0` indigo, `r1` green, `r2` orange, `r3` pink); spilled values have a red dashed outline.
- **Table**: for each value its gain $G(v)$, its bid (the coalition's total), live-range size in blocks, policy-iteration rounds, and its location (a register chip, or a red `slot` chip if spilled). Sorted by bid.
- **Sealed-bid auction rounds** (scroll down): every contested clique in MCS order, with winners in green and losers in red.
- **Allocator menu**: switch to Static-Prior, loop-depth or linear scan to see how their choices differ.

Point out: `i#1.3` has a small gain (12,288) but bids 94,223 because it is in a coalition with `i#1.2`; both get `r1`. Then switch to *Static-Prior* and show that the hot counters lose their registers.

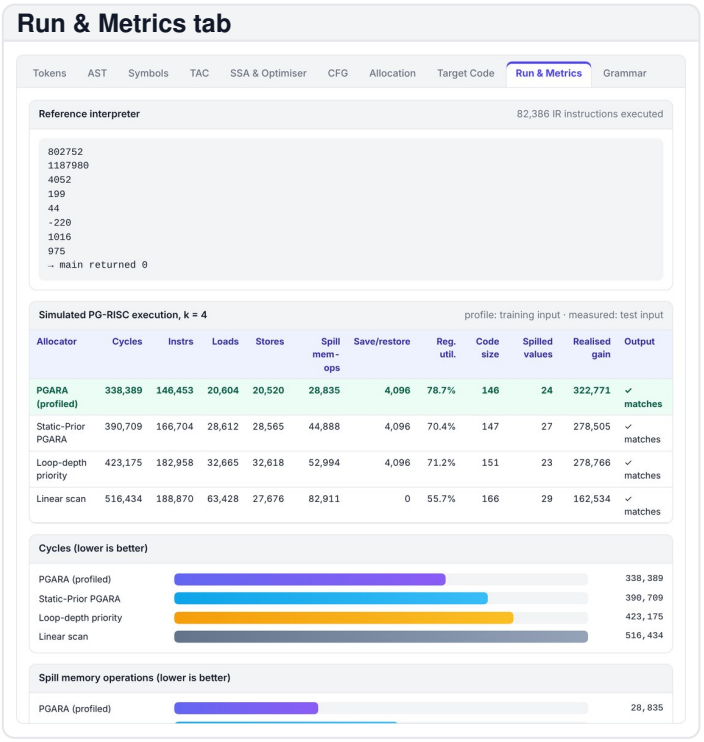
Target Code

Target Code tab, stepping through the assembly



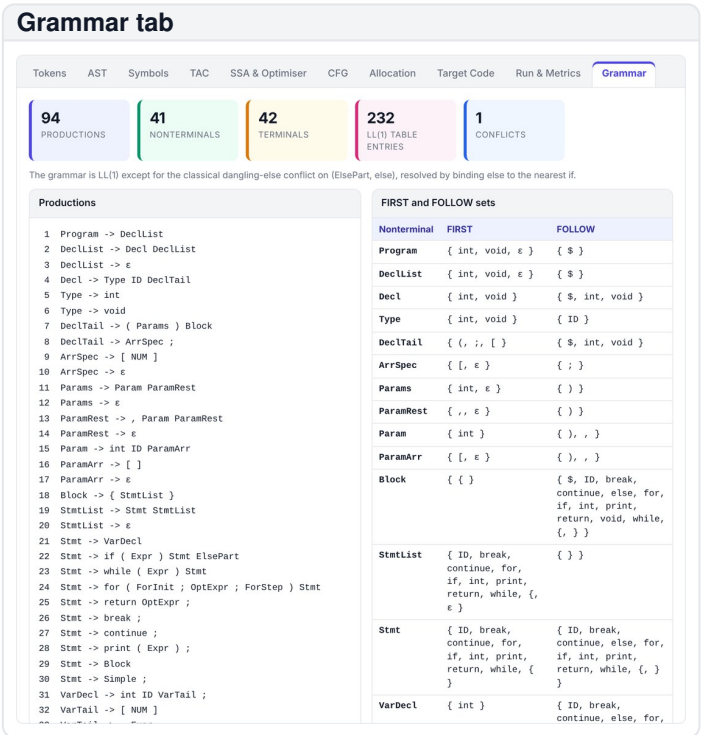
The PG-RISC assembly. Step through it with the buttons; the matching source line is highlighted in the editor. Hover a line to see the IR it came from. Tags: spill loads and stores red, caller-save code amber, φ copies violet, forwarded loads green.

Run & Metrics



The output of the reference interpreter, then a table comparing all four allocators on the test input: cycles, instructions, loads, stores, spill memory operations, save/restore, register utilisation, code size, number of spilled values, realised gain and whether the output matches (it always should). The best row is green. Bar charts follow, in the allocator colours (PGARA indigo, Static-Prior sky blue, loop-depth amber, linear scan grey).

Grammar



The full LL(1) grammar (41 nonterminals, 94 productions), FIRST and FOLLOW sets, and the check of the predictive table, which reports one conflict: the dangling `else`.

7.4 A five-minute demo script

1. Open the website. Choose *Rare error branch*, $k = 4$, training `7 10`, test `13 10`.
2. **CFG tab**: show the hot path in red and an edge where profiled and static probabilities differ.
3. **Allocation tab**: show the gains, the coalition bid of `i`, and the auction rounds. Switch the allocator to Static-Prior and back.
4. **Run & Metrics**: PGARA is fastest; all outputs match.
5. Change the training input to `7 600` (errors become common in training only). Show that PGARA now does worse than Static-Prior on the test. This is the honest limitation: the profile must be representative.
6. Type a small error in the editor to show diagnostics, then fix it.
7. Optionally lower k to 3 or raise it to 8 and watch spilling change.

8 The results and how to read them

8.1 Setup

Ten benchmarks (straight-line code, array sum, matrix multiply, recursion, rare error branch, early-exit search, switch-like dispatch, token scanner, insertion sort with binary search, Collatz with a sieve), five register budgets ($k = 3, 4, 5, 6, 8$), four allocators: 200 compiled programs. Each is profiled on its training input and measured on a different test input. Everything shares the same front end, optimiser and code generator, so only the allocator differs.

8.2 Metrics

Cycles Simulated running time on the test input. The main metric.

Spill memory operations Loads and stores of spill slots actually executed. This is what the gain $G(v)$ tries to avoid.

Register utilisation Share of operand accesses served from registers.

Code size Number of instructions in the program.

Compile cost Policy-iteration rounds and allocation time.

8.3 Key numbers

Geometric-mean cycles vs linear scan	$k = 3$	$k = 4$	$k = 6$	$k = 8$
PGARA (profiled)	**0.813**	**0.795**	**0.793**	**0.777**
Static-Prior PGARA	0.856	0.815	0.819	0.797
Loop-depth priority	0.874	0.850	0.846	0.844
Linear scan	1.000	1.000	1.000	1.000

- PGARA is fastest on average at every register budget.
- At $k = 4$: 59% fewer spill memory operations than linear scan, 21% fewer than loop-depth and 12% fewer than Static-Prior.

- At $k = 8$ PGARA almost removes spill traffic: 4.6 thousand operations against 27.9k (Static-Prior), 72.2k (loop-depth) and 213k (linear scan).
- The auction-based allocators produce code 9 to 17% smaller than linear scan.
- Allocation is cheap: at most 7 policy-iteration rounds for any value, and about 22 ms for all four allocators on the largest program.
- Biggest wins: rare-branch (0.655 of linear scan), dispatch (0.542), matmul (0.692), collatz (0.695). On straight-line code and recursion all allocators are about equal, because there are no branches to learn from.

8.4 Honest limitations (say these before you are asked)

Limitations we state in the paper

- **The profile must be representative.** With a deliberately wrong training input, PGARA falls 2 to 10% behind Static-Prior (it still beats loop-depth and linear scan). The same is true of every profile-guided optimisation.
- **Simulator, not real hardware.** Cycles come from a fixed latency table with no caches.
- **Small benchmarks**, written for this study, and $\theta = 0.25$ was chosen on the same suite (from 0.1, 0.25 and 0.5).
- **Whole live range.** The MDP can say "reload halfway through", but the code generator keeps a value either in a register or in memory for its whole life (no live-range splitting). That part of the policy is computed but not used.
- **More registers is not always strictly better** for every program, because the auction and tie-breaking are local decisions; on average the trend is clear.
- PGARA may spill *more* values than loop-depth (191 against 159 at $k = 4$), but they are the cold ones. That is intended.

8.5 Future work

Use the MDP's policy for live-range splitting; use phase-aware or context-sensitive profiles; port the engine to LLVM to test on standard suites and real hardware; combine measured profiles with learned branch prediction when no training input exists.

9 Glossary

Abstract syntax tree (AST)	A tree showing the structure of the program, without punctuation.
Attribute grammar	A grammar whose symbols carry values (attributes) computed by rules; *inherited* attributes flow down the tree (e.g. the scope), *synthesised* ones flow up (e.g. the type).
Auction (sealed-bid)	Each bidder submits one hidden bid; the highest bids win. Ours awards the k registers of a clique to the k highest bids.
Back edge	A CFG edge that jumps back to a loop's header. Drawn violet and dashed in the GUI.

Basic block	A straight run of instructions with one entry and one exit.
Bellman equation	“Best from here = today’s reward + best expected future.” The equation an optimal MDP solution satisfies.
Best response	Each player changes to its best choice given what others are doing. Repeating this until nobody changes reaches a Nash equilibrium.
Bid $\beta(v)$	What a value offers in the auction: the total gain of its coalition.
Caller-save	Registers in use across a call are saved before the call and restored after it.
CFG	Control-flow graph: blocks as nodes, possible jumps as edges.
Chordal graph	A graph in which every cycle of four or more nodes has a shortcut edge. SSA interference graphs are always chordal.
Clique	A group of nodes all joined to each other; here, values all alive at the same moment.
Coalition	A team of values joined by φ-functions over hot edges that bid together.
Coalition repair	If a team ends up split and the copy cost is higher than the members’ gain, the whole team gives up its registers.
Congestion game	A game where players compete for shared resources (here registers). Such games always have a pure Nash equilibrium.
Dangling else	The ambiguity in `if a if b x else y`: which `if` owns the `else`? We attach it to the nearest one.
DCE	Dead code elimination: removing instructions whose results are never used.
DFA	Deterministic finite automaton: the state machine our lexer is built on.
Dominator	Block a dominates b if every path from the entry to b goes through a.
Edge profile	Counts of how many times each CFG edge was taken during the training run.
FIRST / FOLLOW	Sets of tokens that can start, or come right after, a grammar symbol. Used to build the LL(1) table.
Forwarding	Turning “store a value, then immediately load it back” into a register move.
Gain $G(v)$	Expected cycles saved per call by keeping v in a register instead of memory.
Geometric mean	The average used for ratios: multiply n ratios and take the n-th root.
GVN	Global value numbering: finds and removes repeated identical calculations.
Interference graph	Nodes are values; an edge means the two are alive at the same time and cannot share a register.
Interpreter (reference)	Runs the IR directly; gives the correct output and the profile.
k	The number of registers available to the allocator.
Linear scan	A fast baseline allocator that walks live intervals in order and spills the one that ends last.
Live range	The blocks where a value is still needed later.
Liveness analysis	Computing which values are live at each point of the program.
LL(1)	A grammar that can be parsed top-down choosing each rule from one token of lookahead.
Loop-depth priority	A baseline that weights each use by $10^{\text{loop depth}}$.

Markov decision process (MDP)	A model of decisions under chance: states, actions, rewards and transition probabilities. Ours has two states, register and memory.
Maximal munch	The lexer rule: always take the longest possible token.
MCS	Maximum cardinality search: an order that repeatedly visits the node with the most visited neighbours. On a chordal graph it reveals the cliques.
MiniC	Our small C-like source language (int scalars and arrays, functions, if/while/-for, input/print).
Nash equilibrium	A state where no player can do better by changing only its own choice.
Peephole	Small local clean-ups on the final assembly.
PEO	Perfect elimination ordering: an order in which each node's later neighbours form a clique. Exists exactly for chordal graphs.
PG-RISC	Our simple load/store processor with registers `r0...r(k-1)` and scratch registers `t0`–`t2`.
φ-function	In SSA, picks which version of a variable to use depending on which path arrived.
φ-web	All SSA values connected through φ-functions (the versions of one source variable).
Policy	A rule saying which action to take in every state.
Policy iteration	Howard's exact MDP solver: evaluate a policy, improve it, repeat until it stops changing.
Profile-guided	Using measurements from a real run to guide compilation decisions.
Register utilisation	Share of operand accesses served from registers.
SCCP	Sparse conditional constant propagation: finds constants and unreachable branches together.
Spill / reload	Moving a value from a register to memory (store) / back (load).
SSA	Static single assignment: every variable is assigned exactly once.
Static-Prior PGARA	PGARA with guessed probabilities instead of measured ones; shows how much the profile itself helps.
Symbol table	The compiler's record of every declared name, its type, scope and IR name.
TAC	Three-address code: each instruction has at most one operator and three operands.
θ (hot-edge threshold)	0.25: the share of traffic an edge needs to join two values into a coalition.
Training / test input	Input used for profiling / a different input used for measuring.
Value iteration	Another MDP solver that repeats the Bellman update until values stop changing; we use it only as a cross-check.

10 Likely viva questions, with answers

Q. What does PGARA stand for, and what does each word mean?

Profile-Guided Adaptive Register Allocation. *Profile-guided*: branch probabilities are measured on a training run. *Adaptive*: when measured behaviour changes, gains, bids and therefore the allocation change too. *Register allocation*: choosing which values live in registers.

Q. How is this different from graph colouring?

We never colour the graph. The interference graph is only used to find the cliques where values compete. The decision of who gets a register comes from the expected gain of each value (an MDP) and an auction, not from colourability.

Q. Why an MDP and not just a count of uses?

A count ignores that a value can be spilled and reloaded, that crossing a call costs a save and restore, and that some uses are on rare paths. The MDP adds all of that up as an expected value under the measured probabilities, and its optimal policy tells us the best register/memory behaviour.

Q. Why policy iteration and not value iteration?

Policy iteration gives the exact answer in a finite number of steps (1 to 7 here) by solving a linear system each round. Value iteration only converges approximately. We keep value iteration in the tests to confirm both agree.

Q. Why is there no discount factor?

A program always leaves a live range eventually, so the total reward is finite (a transient, total-reward MDP). A discount would distort the cycle counts.

Q. What is $G(v)$ measured in?

Expected cycles saved per call of the function, using the same costs as the simulator (load 4, store 2, register access 1).

Q. Why do φ-related values form coalitions?

If one version of a loop variable got a register and the next did not, every loop trip would copy through memory. Bidding the team total makes them win or lose together. Only hot edges join teams, so a cold copy does not drag a cold value into a hot team.

Q. Why does chordality matter?

In a chordal graph visited in MCS order, a node's visited neighbours form a clique, a set of values alive together. That is exactly where register pressure exceeds k, so the auction is held there. SSA guarantees chordality, and we verify it on every function.

Q. Where is the game theory?

Two places. The auction (sealed bids, top k win) and the coalition game, where best-response dynamics reach a pure Nash equilibrium: no spilled coalition can take free registers, and no winner can improve.

Q. Is the Nash equilibrium the global optimum?

Not necessarily; congestion games have many equilibria. The auction steers us to a high-gain one, and best response only fills registers the auction left free.

Q. How do you know the compiler is correct?

Every one of the 200 runs is checked against the reference interpreter's output, and 23 unit tests cover every phase, including a hand-computed gain and the Nash property.

Q. What happens if the training input is unrepresentative?

PGARA can lose to Static-Prior by 2 to 10% (Table of sensitivity in the paper), though it still beats loop-depth and linear scan. That is a property of all profile-guided optimisation; Static-Prior is the fallback when no good training input exists.

Q. Why not compare with LLVM?

PG-RISC is our own target and not an LLVM back end. Comparing inside one compiler keeps everything except the allocator identical, so the comparison is fair. Porting to LLVM is future work.

Q. How long does allocation take?

Milliseconds: about 22 ms for all four allocators on the largest program, in pure Python.

Q. What is the time complexity?

Per value, policy iteration solves a linear system the size of its live range (at most a few rounds); MCS is linear in the graph size; each auction round sorts one clique of at most ω values; best response makes at most $|V|$ moves.

Q. What was each member's contribution?

See the team table on the cover: front end and TAC (Shreyans), semantic analysis (Samrender), optimiser, allocator and code generation (Karthikeya), runtime, tests and benchmarks (Abhishek), website and documentation (Nishanth).

11 Map of the research paper

Abstract, keywords	The problem, the method, the main numbers.
I. Introduction	Why register allocation matters, why static guesses fail on data-dependent branches, our contributions.
II. Related Works	The fifteen recent papers in five themes, each with its gap (Section 5 of this guide).
III-A. Problem formulation	Allocation as maximising total expected gain under the register limit.
III-B. System architecture	The pipeline diagram (Fig. 1 of this guide).
III-C to III-F. Front end	Grammar, LL(1) check, parser, attribute grammar for semantics, TAC and jumping code, CFG, SSA, optimiser.
III-G to III-L. PGARA	Profiling, the MDP, gain, coalitions, auction, best response (Section 3 here), with Algorithms 1–3.
III-M. Back end	Out-of-SSA, spill slots, peephole, simulator.
III-N. Modules	Who built which part.
IV. Implementation and results	Tools, parameters, benchmarks, baselines, metrics, cycle chart, spills, utilisation, code size, sensitivity, comparison table, threats to validity, and the website screenshots.

V. Conclusion	Summary and future work.
End matter	Author contributions, acknowledgements, funding, competing interests, data and code availability, references.

If you remember only three things

1. PGARA **measures** branch probabilities instead of guessing them.
2. Each value's register worth is an exact **expected saving** $G(v)$ from a small MDP solved by policy iteration.
3. Values (in φ-web teams) **bid** that saving in an **auction** on the cliques of the chordal interference graph, and best response settles it into a **Nash equilibrium**. No graph colouring anywhere.